Competitive Exam Practice 8 — Answer Key
I/O Techniques

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

August 12, 2026

Q1 (D). The CPU never reacts mid-instruction: the lecture's five-step scenario (step 2) states it finishes its *current instruction* first, then checks `PS.IE`, then branches to the ISR. (A) and (B) describe behaviours no interrupt should cause; (C) omits the finish-the-instruction condition.

Q2 (B). In a vectored interrupt each device supplies its own identifying code, and hardware uses it to index the **interrupt-vector table** — a table of ISR starting addresses, one entry per device — jumping straight to the correct ISR. (A) describes a fixed vector location (not device-supplied), (C) describes polling a processor register, and (D) is wrong because (B) holds.

Q3 (A). The lecture's two-switch gating makes this precise: a request reaches the CPU only when the device's enable *and* `PS.IE` both allow it, so with interrupts disabled the CPU will not process them. (C) is false as stated in GATE's intended sense — the CPU checks for interrupts *after* completing an instruction, i.e. between instructions, not "before executing a new instruction" in the way (C) claims to be the distinguishing truth; (B) and (D) are plainly false (interruptible loops exist; edge-triggered interrupts exist).

Q4 (A). In a daisy chain, the bus-grant (or interrupt-acknowledge) signal propagates device-to-device in a fixed physical order, so the device closest to the processor is granted first — priority is non-uniform (highest for the nearest), not uniform (B), and it needs only one shared signal path, not a separate pin per device (D).

Q5 TRUE. A bus (DMA) request outranks an interrupt request: the DMA/bus request is the lower-latency, hardware-initiated path that must be served before the interrupt can even be processed (the interrupt itself needs the bus), and it cannot be delayed without risking loss of the on-going transfer. This matches the lecture's ordering of techniques, where DMA sits *below* the interrupt layer on the critical path.

Q6 TRUE. In memory-mapped I/O the device registers live in the normal address space, so *any* memory-referencing instruction (including block-move instructions) can be used to transfer data; I/O-mapped I/O is restricted to dedicated `IN`/`OUT` instructions. More flexible, direct addressing — no special instruction decode, no narrower address range — makes data transfer between microprocessor and device *usually* faster, which is the sense the statement asserts.

Q7 TRUE. Programmed (polling) I/O, when the device is already ready, performs the data transfer directly — a single read/write of the data register. Interrupt-driven I/O always carries the overhead of the full five-step handling scenario on top of the same transfer: finish

the current instruction, save `PC` (and shadow registers), disable `IE`, jump to the ISR, service the device, and execute `RTI` to restore state and re-enable interrupts. That per-transfer overhead is why programmed I/O is faster *per transfer* when the device is ready, exactly the lecture's poll-vs-interrupt trade-off table: polling's cost is the wasted waiting, not the transfer itself.

Q8 **100000000** (10^8). One keystroke every 0.1 s; at 2×10^9 instructions/s the CPU executes $0.1\times 2\times10^9 = 2\times10^8$ instructions between keystrokes; at 2 instructions per failed iteration, that is $2\times10^8 \div 2 = 10^8$ iterations — each one re-reading an unchanged status flag, accomplishing nothing useful.

Q9 **(B).** The synchronous/async trade-off table is decisive here: asynchronous pays *two full round-trip delays* per transfer (steps 1–2 and 3–4 of the handshake), which a synchronous bus never pays for a consistently fast device; synchronous only loses on slow devices (wait states) or when the clock period must shrink to beat skew. For devices of known, similar speed, the cheaper synchronous protocol wins.

Q10 **(B).** Cycle stealing is the DMA controller temporarily taking memory bus cycles the CPU would otherwise have used, because both compete for the same shared memory bus — the CPU is delayed only for the individual stolen cycles, not halted. (C) inverts the whole point of DMA (the CPU is *not* running one ISR per word), and (A)/(D) get the direction of the competition backwards.

Q11 **(B).** Where DMA offloads *moving* data (one pre-configured block transfer, then stop), an IOP is a dedicated processor given a whole *channel program* — several transfers, data formats, and simple decisions — that it executes independently, interrupting the main CPU only when the entire program completes. (A), (C), and (D) contradict the lecture: an IOP is *more* hardware than a DMA controller, does interrupt the CPU (once, at program completion), and must still arbitrate for the bus when it moves data.